Why we chose Problems 9 and 11

Hackathon project decision report | 17 October 2026

We have chosen **Problem 9 — Medication Safety** and **Problem 11 — Smart Data Masking** as our two planned entries. Both address practical problems, can produce a clear demonstration, and give us concrete ways to check whether our solution works.

This is our project choice, not confirmation that the applications have been submitted. We still need clarification on the applicable template, permitted AI tools and when implementation may begin.

What we investigated and why

Before committing, we checked the event information, the signed-in portal, all 12 problem statements, their supporting documents and the available API response. We wanted to understand what the organizers actually require and avoid choosing a project based only on an AI assistant's confident suggestions.

The investigation established three useful things:

- **The written deliverables give us a better basis for choosing than guesses about competitors.** We found no per-problem applicant counts. We cannot claim that either choice has fewer competing teams.
- **There is a rules conflict to resolve.** Most problems link to a seven-slide template, while another official template has six slides and restrictions on pre-built code, computing resources and data/model access. We cannot assume those restrictions exempt our projects.
- **Some earlier AI advice was overstated.** We did not establish the private screening prompt, guaranteed winning strategies or unrestricted permission to use our own AI services. Draft performance numbers were not measured results.

Our advantage should come from meeting the requirements convincingly, rather than guessing how to manipulate the screening process.

Why these two

We kept **Problem 11** because it has a visible, testable outcome: different users receive different views of sensitive data, while the original records stay unchanged.

We replaced **Problem 7 — Discharge Care Plans** with **Problem 9** because we judge its core checks to be more controllable for our team. Problem 7 requires preserving medical instructions while simplifying and translating them. A fluent-looking result can still contain a dangerous omission or translation error.

Problem 9 instead asks us to check medication lists against reference tables supplied for the exercise. Those tables give us a defined basis for checking many results. This does not make Problem 9 easy: recognizing misspelled medicine names and avoiding unnecessary warnings are central challenges.

The teammate originally suggested Problem 9. Our independent review supports that suggestion for these reasons, rather than because it sounds more impressive or is assumed to attract fewer competitors.

Problem 9 A second check for medication risks

The pitch: A clinician enters a patient's medication list. Our tool identifies potential problems, puts the most serious ones first, and explains what needs review.

We would first match brand names and misspellings to the medicines in the supplied reference. Then we would check for interactions, duplicate treatments and relevant patient-specific restrictions. Each alert would show the medicines involved, its severity, a suggested action supported by the reference, and any uncertainty.

The clinician remains the decision-maker. The tool must not declare a patient “safe” simply because it found no issue in an incomplete list.

What proves it works: correctly matching medicine names, detecting serious issues and the supplied near-miss cases, and avoiding a flood of low-value alerts. These are explicit parts of the official problem.

Main difficulties: ambiguous names, incomplete inputs, interpreting the supplied rules and balancing missed risks against excessive warnings. We need the promised reference tables and evaluation assets; they have not yet been obtained here.

Problem 11 Sensitive data visible only as permitted

The pitch: A bank's support worker should see enough information to help a customer without receiving all their sensitive details. Our tool automatically finds sensitive fields and returns an appropriate view for each authorized role.

We would demonstrate this on two databases, provisionally PostgreSQL and SQLite. An administrator sees full values, support sees partial values, and an analyst receives redacted or transformed values according to a defined policy. The original database records remain unchanged.

The protection must happen before restricted data reaches the user. Simply covering a number on screen would not be enough.

What proves it works: sensitive fields are detected, each role receives the expected values on both databases, unauthorized requests are rejected, and the original records remain intact.

Main difficulties: incorrectly identifying sensitive fields, trusting a user-selected role, and allowing a complicated query to expose information unexpectedly. We will begin with a limited set of supported queries and test them carefully instead of promising to handle every possible database operation.

How we would work

Irfaan would coordinate the technical build using permitted AI assistance. The teammate could help own example cases, requirement checks, understandable explanations and demonstration testing. We should agree that division together; both members need to understand the final demonstration.

For each project, the sequence is: understand the supplied inputs, define expected results, build one complete workflow, test failures, then improve the presentation. Extra features come after the core works.

We should also confirm whether two shortlisted applications would require choosing one project for the finale. Preparing two entries does not mean we have established that building both within the event is feasible.

How likely are we to get it right

A focused prototype is a credible goal; a dependable solution is not automatic. AI can accelerate implementation, but checking its output is still our responsibility.

- **Explain each idea clearly and prepare a credible proposal:** Achievable with a narrow scope and the correct template
- **Build a working core demonstration:** Plausible if tools are permitted, inputs arrive and we keep the scope small
- **Perform well on unfamiliar evaluation cases:** Unproven until we test on representative data and the available harness
- **Build two dependable projects during the finale:** A substantial workload risk for a two-person team
- **Get shortlisted or win:** No defensible probability yet; competitors and judging outcomes are unknown

Our main risks are unresolved rules, limited time, unfamiliar data and accepting AI-generated code without understanding its failures. We reduce those risks by testing early, keeping features limited and making both the strengths and limitations visible.

What we gain if we execute well

We can show working evidence of solving a real problem, rather than only an attractive interface. Medication Safety can demonstrate useful prioritization with understandable explanations. Data Masking can demonstrate enforced permissions across two databases.

Those are concrete technical stories we can explain to judges and potentially use as portfolio evidence, subject to the event's ownership and publication rules. They do not guarantee prizes, jobs or commercial readiness.

The immediate next step is to settle the rules and team/application details, then define the smallest convincing demonstration for each entry. More speculation about applicant counts will not improve that plan.

Sources

Official InnoSmart event information and portal documents, reviewed on 17 October 2026. The practical recommendations and feasibility assessment are our judgment, not organizer guarantees.

Event page: https://innosmart.in/ics/2026/MegaHackathon-Oct26/

Problem statements: H4 The Medication Safety Second Pair of Eyes; Smart Data Masking System for PII Protection. Available through the competition portal: https://app.innosmart.in/rp/competitions/13